

Solucionador de Sudoku: Implementação Serial e Paralela com OpenMP

Carlos Tchípia, Emanuel dos Santos, Lúria Bá
Engenharia Informática, ISPTec — Luanda, Angola
Computação Paralela e Distribuída, 2025/2026

Abstract—Este artigo apresenta a implementação e análise de um solucionador do quebra-cabeça Sudoku em linguagem C, nas versões serial e paralela com OpenMP. A versão serial utiliza backtracking recursivo com verificação $O(1)$ por bitmasks. A versão paralela explora o paralelismo através da decomposição do espaço de busca em múltiplos níveis ($K=3$), gerando até 1024 sub-problemas distribuídos dinamicamente pelas threads. Apresentam-se os tempos de execução para 1, 2 e 8 threads na instância 16×16 , os speedups e a eficiência paralela. Os resultados revelam degradação de desempenho com o aumento de threads, cujas causas — overhead de sincronização, desequilíbrio de carga e contenção — são discutidas em detalhe.

Keywords—Sudoku; backtracking; bitmask; OpenMP; decomposição do espaço de busca; sub-problemas; speedup; balanceamento de carga.

I. INTRODUÇÃO

O Sudoku é um problema combinatório cujo espaço de estados cresce exponencialmente com a dimensão da grelha. Para grelhas 16×16 com numerosas células vazias, a resolução serial pode exigir vários segundos, tornando este problema num caso de estudo natural para técnicas de computação paralela.

Este trabalho insere-se na disciplina de Computação Paralela e Distribuída (CPD) do ISPTec, ano lectivo 2025/2026 (Grupo 4). O projecto compreende três fases: V0 (implementação serial com backtracking), V1 (paralelização com OpenMP para arquitecturas UMA) e V2 (distribuição com MPI). Este relatório cobre as fases V0 e V1, com foco na análise de desempenho da versão paralela com bitmasks e geração de sub-problemas em múltiplos níveis.

II. DESCRIÇÃO DO PROBLEMA

O puzzle consiste numa matriz parcialmente preenchida $n \times n$, onde $n=L^2$. O objectivo é preencher as células a zero com valores de 1 a n , respeitando: (i) cada linha contém exactamente uma ocorrência de cada valor; (ii) cada coluna contém exactamente uma ocorrência de cada valor; (iii) cada bloco $L \times L$ contém exactamente uma ocorrência de cada valor.

A estrutura central é Sudoku, que contém a grid $n \times n$, as bitmasks `row_mask`, `col_mask` e `box_mask` (actualizadas incrementalmente) e os parâmetros n e L . Um bit k activo em `row_mask[i]` indica que o valor k já está presente na linha i . A verificação de validade reduz-se a uma operação $O(1)$: $(\text{row} \mid \text{col} \mid \text{box}) \& (1 \ll \text{num}) == 0$.

III. IMPLEMENTAÇÃO SERIAL (V0)

A. Algoritmo de Backtracking com Bitmasks

O núcleo do solucionador serial é `solve_with_cutoff`, que implementa backtracking recursivo com verificação de validade $O(1)$ via bitmasks. Para cada célula vazia encontrada em ordem de leitura (row-major), calcula-se a máscara de valores já usados (`used = row | col | box`) e testam-se apenas os valores cujo bit não está activo. Ao colocar um valor, o bit correspondente é activado nas três máscaras; ao fazer backtrack, é desactivado via XOR.

Esta abordagem elimina os três loops da versão `is_valid` original ($O(n)$ por verificação) e substitui-os por operações de bits de custo constante, resultando numa aceleração significativa da versão serial face a implementações naïves.

B. Medição de Tempo e Validação

Conforme especificado no enunciado, o tempo de execução é medido com `omp_get_wtime()` (wall-clock time), contabilizando apenas o algoritmo e excluindo I/O. O resultado é enviado para `stderr`; a solução para `stdout`. A correcção foi verificada em instâncias 4×4 , 9×9 e 16×16 , com e sem solução.

IV. IMPLEMENTAÇÃO PARALELA COM OPENMP (V1)

A. Estratégia de Decomposição em Múltiplos Níveis

A estratégia adoptada é a decomposição do espaço de busca por geração de sub-problemas em $K=3$ níveis da árvore de backtracking (função `generate_tasks`). Em vez de paralelizar apenas no primeiro nível (máximo n tarefas, potencialmente insuficiente para muitas threads), a expansão a K níveis gera até $n^K = 9^3 = 729$ sub-problemas para $n=9$, ou até $\text{MAX_TASKS}=1024$ sub-problemas para qualquer dimensão. Isto garante trabalho suficiente independentemente do número de threads.

Cada sub-problema é um `SubProblem` independente: uma cópia completa da grid e das bitmasks no estado parcialmente preenchido após K decisões. Esta independência elimina partilha de estado mutável durante o backtracking, tornando desnecessárias secções críticas no caminho quente.

B. Paralelização e Terminação Antecipada

Os sub-problemas são distribuídos pelas threads com `#pragma omp parallel for schedule(dynamic, 1)`. O scheduling dinâmico acomoda a variabilidade no custo de cada sub-árvore, que depende das restrições já satisfeitas. Quando uma thread encontra a solução, actualiza a `flag found` via `#pragma omp atomic write`; as restantes verificam `*found` no início de cada iteração e terminam antecipadamente.

A cópia da solução para o buffer partilhado `solution` é protegida por `#pragma omp critical`, evitando escritas concorrentes sem overhead no caminho quente (executado no máximo uma vez).

C. Sincronização e Correcção

As únicas operações partilhadas entre threads são: (i) leitura/escrita atómica de `found`; (ii) cópia protegida por `critical` da solução final. Todo o backtracking opera sobre dados privados por thread, sem condições de corrida. A correcção é garantida pelo facto de cada sub-problema representar um estado consistente (bitmasks correctas) gerado sequencialmente antes da fase paralela.

V. RESULTADOS EXPERIMENTAIS

A. Ambiente de Teste

Os testes foram executados na instância 16×16 (ficheiro `16x16.txt`). O tempo de execução serial (versão `sudoku-serial`, 1 thread) foi de 15,1 s. A versão paralela foi testada com $\text{OMP_NUM_THREADS} \in \{1, 2, 8\}$. Compilação com `gcc -fopenmp -O2`. O speedup é calculado como $S(p) = T_{\text{serial}} / T_p$ e a eficiência como $E(p) = S(p) / p$.

B. Tempos de Execução — Instância 16×16

Threads	Tempo (s)	Speedup S(p)	Eficiência (%)
Serial (ref.)	15,1	1,00×	100,0
1	4,2	3.60×	359.5
2	6,4	2.36×	118.0
8	25,2	0.60×	7.5

Tabela: Tempos de execução, speedup e eficiência para a instância 16×16 . $S(p) = T_{\text{serial}} / T_p$; $E(p) = S(p) / p \times 100$. A linha a vermelho indica degradação face ao serial.

VI. DISCUSSÃO

A. Análise dos Resultados

Os resultados revelam um padrão anti-intuitivo: a versão paralela com 1 thread (4,2 s) é 3,6× mais rápida que o serial (15,1 s), confirmando que as optimizações de bitmask $O(1)$ introduzidas na versão OMP são genuinamente mais eficientes que a versão serial original. Contudo, com 2 threads (6,4 s) e 8 threads (25,2 s), o desempenho degrada progressivamente — o tempo aumenta em vez de diminuir.

Esta degradação indica que o overhead de paralelização supera os ganhos de paralelismo. Os factores determinantes são: (i) criação e gestão de até 1024 sub-problemas, cada um com cópia completa da grid e bitmasks (alocações de memória n^2+3n inteiros por sub-

problema); (ii) contenção na flag found (lida em cada nível de recursão por todas as threads); (iii) desequilíbrio de carga severo — sub-árvores com poucas restrições podem ser ordens de grandeza mais custosas que outras, fazendo algumas threads esperarem enquanto outras terminam; (iv) degradação de cache causada pelo acesso de múltiplas threads a diferentes regiões de memória.

B. Comparação com Expectativas

Esperava-se speedup próximo de p para instâncias com trabalho abundante. Na prática, o overhead de geração e distribuição de sub-problemas é demasiado elevado para esta instância, na máquina de teste. Para puzzles mais difíceis (e.g., grelha 16×16 completamente vazia), onde cada sub-árvore exige muito mais trabalho, o overhead relativo seria menor e os speedups poderiam ser positivos. O desequilíbrio de carga, inerente ao backtracking, é o factor limitante mais relevante.

C. Balanceamento de Carga

O `schedule(dynamic, 1)` distribui os sub-problemas às threads à medida que ficam disponíveis, mitigando o desequilíbrio estático. Contudo, quando alguns sub-problemas são ordens de grandeza mais custosos (e.g., sub-árvore com poucas restrições vs. sub-árvore rapidamente podada), uma única thread pode monopolizar o tempo total, deixando as restantes ociosas após terminarem os seus sub-problemas mais fáceis. A terminação antecipada via found reduz este desperdício no caso com solução, mas não elimina o desequilíbrio.

VII. CONCLUSÃO

Este trabalho apresentou a implementação de um solucionador de Sudoku em C nas versões serial (V0) e paralela com OpenMP (V1). A versão serial adoptou verificação $O(1)$ por bitmasks, resultando num desempenho substancialmente superior a implementações com `is_valid` iterativo. A versão paralela gerou sub-problemas em $K=3$ níveis para garantir granularidade suficiente, distribuídos com `schedule(dynamic, 1)`.

Os resultados experimentais na instância 16×16 evidenciaram que a versão com 1 thread OMP (com bitmasks) é $3,6\times$ mais rápida que o serial original, mas que o aumento de threads causa degradação — o overhead de criação de sub-problemas e a contenção superam os ganhos de paralelismo para esta instância e configuração. Como trabalho futuro (V2), será implementada uma versão MPI com decomposição distribuída do espaço de busca e balanceamento dinâmico de carga entre processos.

AGRADECIMENTOS

Os autores agradecem ao Docente João da Costa pelo acompanhamento e clarificações sobre o enunciado do projecto.

REFERÊNCIAS

- [1] B. Chapman, G. Jost, R. van der Pas,
Using OpenMP: Portable Shared Memory Parallel Programming.
MIT Press, 2008.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein,
Introduction to Algorithms, 4th ed. MIT Press, 2022.
- [3] Enunciado do Projecto — Solucionador do Sudoku (V1.0). CPD, ISPTEC, 15/03/2026.